

LAPORAN PRAKTIKUM

MATERI 6: DEADLOCK



**Dosen Pengampu :
Triawan Adi Cahyanto M.Kom**

**Disusun Oleh :
MUHAMMAD RIDHO 2410651009 (A)**

Tanggal : 17-11-2025

**PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025**

1. Analisis Deadlock

- Deadlock sederhana

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Deklarasi dua mutex (seperti kunci pintu)
pthread_mutex_t lock1 =
PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 =
PTHREAD_MUTEX_INITIALIZER;

// Fungsi untuk Thread 1
void* thread1_func(void* arg) {
    printf("Thread 1: Mencoba mengambil Lock 1...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 1: Berhasil dapat Lock 1!\n");

    sleep(1); // Simulasi proses kerja

    printf("Thread 1: Mencoba mengambil Lock 2...\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 1: Berhasil dapat Lock 2!\n");

    printf("Thread 1: Selesai bekerja\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

// Fungsi untuk Thread 2
void* thread2_func(void* arg) {
```

```

    printf("Thread 2: Mencoba mengambil Lock 2...\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 2: Berhasil dapat Lock 2!\n");

    sleep(1); // Simulasi proses kerja

    printf("Thread 2: Mencoba mengambil Lock 1...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 2: Berhasil dapat Lock 1!\n");

    printf("Thread 2: Selesai bekerja\n");

    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);

    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    printf("=== SIMULASI DEADLOCK ===\n\n");

    // Membuat dua thread
    pthread_create(&thread1, NULL, thread1_func, NULL);
    pthread_create(&thread2, NULL, thread2_func, NULL);

    // Menunggu thread selesai (tidak akan pernah selesai
    karena deadlock)
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("\n=== PROGRAM SELESAI ===\n");
    return 0;
}

```

- **Mengapa program mengalami deadlock?**

Program mengalami deadlock karena:

- **Thread 1** mengunci **Lock 1**, lalu menunggu **Lock 2**.
- **Thread 2** mengunci **Lock 2**, lalu menunggu **Lock 1**.

Akibatnya:

- Thread 1 tidak bisa lanjut karena Lock 2 sedang dipegang Thread 2.
- Thread 2 tidak bisa lanjut karena Lock 1 sedang dipegang Thread 1.

Kedua thread saling menunggu dan tidak ada yang bisa melanjutkan eksekusi - keadaan mandek (deadlock).

- **Kondisi deadlock apa saja yang terpenuhi?**

Deadlock terjadi jika 4 kondisi Coffman terpenuhi. Pada program ini semuanya terpenuhi:

✓ **1. Mutual Exclusion (Saling mengunci)**

Resource (Lock 1 dan Lock 2) hanya bisa digunakan satu thread pada satu waktu.

✓ **2. Hold and Wait (Menahan dan menunggu)**

- Thread 1 sudah memegang Lock 1, lalu menunggu Lock 2.
- Thread 2 sudah memegang Lock 2, lalu menunggu Lock 1.

✓ 3. No Preemption (Tidak dapat direbut paksa)

Mutex tidak bisa dicabut secara paksa; hanya thread pemiliknya yang dapat melepaskannya.

✓ 4. Circular Wait (Menunggu dalam lingkaran)

Terjadi lingkaran tunggu:

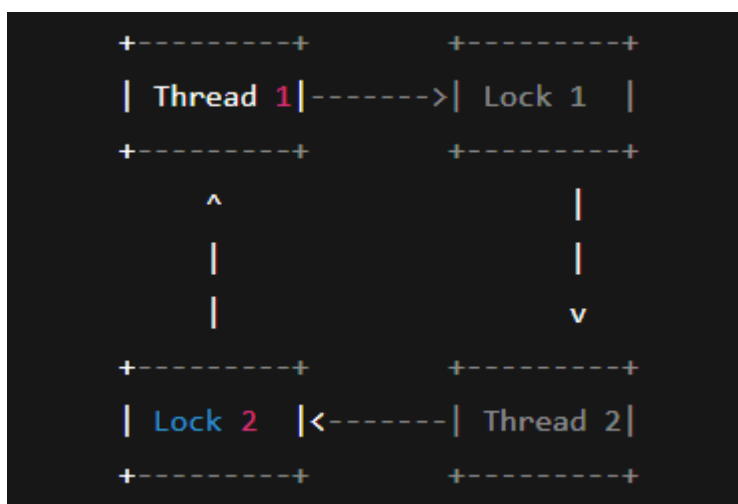
- Thread 1 → menunggu Lock 2
- Thread 2 → menunggu Lock 1

Inilah penyebab deadlock.

2. Diagram Resource Allocation Graph (RAG)

Penjelasan:

- Thread 1 memegang Lock 1 → meminta Lock 2
- Thread 2 memegang Lock 2 → meminta Lock 1
- Terjadi lingkaran (cycle) → indikasi deadlock



Terlihat jelas **cycle** antara thread dan resource → deadlock tidak bisa dihindari.

2. Implementasi Pencegahan

- Teknik Lock Ordering (Urutan Penguncian)

Prinsipnya: semua thread harus mengunci resource dengan urutan yang sama

Misalnya selalu kunci Lock 1 → Lock 2

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
// Function untuk mengunci sesuai urutan yang sama
```

```
void lock_in_order(pthread_mutex_t* first, pthread_mutex_t*  
second) {
```

```
    pthread_mutex_lock(first);
```

```
    sleep(1); // simulasi kerja
```

```
    pthread_mutex_lock(second);
```

```
}
```

```
void* thread1_func(void* arg) {
```

```
    printf("Thread 1: Mengambil Lock 1 -> Lock 2\n");
```

```
    lock_in_order(&lock1, &lock2);
```

```
    printf("Thread 1: Selesai bekerja\n");
```

```
pthread_mutex_unlock(&lock2);  
pthread_mutex_unlock(&lock1);  
return NULL;  
}
```

```
void* thread2_func(void* arg) {  
    printf("Thread 2: Mengambil Lock 1 -> Lock 2 (urutan  
sama)\n");  
    lock_in_order(&lock1, &lock2);  
  
    printf("Thread 2: Selesai bekerja\n");  
  
    pthread_mutex_unlock(&lock2);  
    pthread_mutex_unlock(&lock1);  
    return NULL;  
}
```

```
int main() {  
    pthread_t t1, t2;  
  
    printf("\n=== LOCK ORDERING (Deadlock Prevented)  
===\n");  
  
    pthread_create(&t1, NULL, thread1_func, NULL);  
    pthread_create(&t2, NULL, thread2_func, NULL);
```

```

pthread_join(t1, NULL);

pthread_join(t2, NULL);


return 0;

}

```

Deadlock hilang karena kedua thread mengunci resource dengan urutan yang sama.

```

root@RIDHO:~# nano deadlock_lock_ordering.c
root@RIDHO:~# gcc deadlock_lock_ordering.c -o deadlock_lock_ordering
/usr/bin/ld: /tmp/cc1itHvX.o: in function 'main':
deadlock_lock_ordering.c:(.text+0x12d): undefined reference to 'pthread_create'
/usr/bin/ld: deadlock_lock_ordering.c:(.text+0x14a): undefined reference to 'pthread_create'
/usr/bin/ld: deadlock_lock_ordering.c:(.text+0x15b): undefined reference to 'pthread_join'
/usr/bin/ld: deadlock_lock_ordering.c:(.text+0x16c): undefined reference to 'pthread_join'
collect2: error: ld returned 1 exit status
root@RIDHO:~# gcc deadlock_lock_ordering.c -o deadlock_lock_ordering -lpthread
root@RIDHO:~#
root@RIDHO:~# ./deadlock_lock_ordering

=== LOCK ORDERING (Deadlock Prevented) ===
Thread 2: Mengambil Lock 1 -> Lock 2 (urutan sama)
Thread 1: Mengambil Lock 1 -> Lock 2
Thread 2: Selesai bekerja
Thread 1: Selesai bekerja

```


- Teknik Try-Lock dengan Retry (Non-Blocking Lock)

Thread mencoba mengunci;

Jika gagal → melepas semua lock → menunggu sebentar → mencoba lagi.

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* thread_func(void* arg) {
```

```
    int id = *(int*)arg;
```

```
    while (1) {
```

```
        printf("Thread %d: mencoba lock1...\n", id);
```

```
        if (pthread_mutex_trylock(&lock1) == 0) {
```

```
            printf("Thread %d: berhasil lock1, mencoba lock2...\n", id);
```

```
            if (pthread_mutex_trylock(&lock2) == 0) {
```

```
                printf("Thread %d: berhasil lock2, bekerja...\n", id);
```

```
                sleep(1);
```

```

        pthread_mutex_unlock(&lock2);
        pthread_mutex_unlock(&lock1);

        printf("Thread %d: selesai.\n", id);
        break;
    } else {
        printf("Thread %d: gagal lock2, melepas lock1 & retry.\n",
id);
        pthread_mutex_unlock(&lock1);
    }
}

usleep(200000); // sleep 0.2 detik sebelum retry
}

return NULL;
}

int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    printf("\n=== TRY-LOCK WITH RETRY (Deadlock Avoided)
===\n");

```

```

pthread_create(&t1, NULL, thread_func, &id1);
pthread_create(&t2, NULL, thread_func, &id2);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

return 0;
}

```

Deadlock tidak terjadi, karena jika gagal mengunci kedua resource, thread akan melepas dan mencoba ulang.

3. Perbandingan Kedua Metode

Metode	Cara Kerja	Keunggulan	Kelemahan
Lock Ordering	Semua thread harus mengunci resource dengan urutan yang sama	Sederhana, efisien, tanpa retry	Tidak fleksibel jika resource banyak atau dinamis
Try-Lock + Retry	Mencoba lock; jika gagal → lepas & ulang	Sangat fleksibel, cocok sistem dinamis	Lebih lambat karena banyak retry, CPU usage naik

```
root@RIDH0:~# nano deadlock_trylock_retry.c
root@RIDH0:~# gcc deadlock_trylock_retry.c -o deadlock_trylock_retry -pthread
root@RIDH0:~# ./deadlock_trylock_retry
```

```
=== TRY-LOCK WITH RETRY (Deadlock Avoided) ===
```

```
Thread 1: mencoba lock1...
Thread 1: berhasil lock1, mencoba lock2...
Thread 1: berhasil lock2, bekerja...
Thread 2: mencoba lock1...
Thread 2: mencoba lock1...
Thread 2: mencoba lock1...
Thread 2: mencoba lock1...
Thread 2: mencoba lock1...
Thread 1: selesai.
Thread 2: mencoba lock1...
Thread 2: berhasil lock1, mencoba lock2...
Thread 2: berhasil lock2, bekerja...
Thread 2: selesai.
```

3. Banker algoritma

```
#include <stdio.h>
```

```
#define P 4 // jumlah proses
```

```
#define R 3 // jumlah resource
```

```
int available[R] = {5, 4, 3}; // Available awal
```

```
int max[P][R]; // Matriks Maksimum
```

```
int alloc[P][R]; // Matriks Alokasi
```

```
int need[P][R]; // Matriks Need
```

```
// Mengecek apakah sistem dalam keadaan aman
```

```
int isSafe() {
```

```
    int work[R], finish[P] = {0};
```

```
    for (int i = 0; i < R; i++)
```

```
        work[i] = available[i];
```

```
    int count = 0;
```

```
    while (count < P) {
```

```
        int found = 0;
```

```
        for (int i = 0; i < P; i++) {
```

```
            if (!finish[i]) {
```

```
                int canAllocate = 1;
```

```
for (int j = 0; j < R; j++) {  
    if (need[i][j] > work[j]) {  
        canAllocate = 0;  
        break;  
    }  
}
```

```
if (canAllocate) {  
    for (int j = 0; j < R; j++)  
        work[j] += alloc[i][j];
```

```
    finish[i] = 1;
```

```
    found = 1;
```

```
    count++;
```

```
}
```

```
}
```

```
}
```

```
if (!found)
```

```
    return 0; // Tidak aman
```

```
}
```

```
return 1; // Aman
```

```
}
```

```

// Fungsi request resource
int requestResource(int p, int req[]) {
    // Cek apakah request <= need
    for (int j = 0; j < R; j++) {
        if (req[j] > need[p][j]) {
            printf("Error: Request melebihi need!\n");
            return 0;
        }
    }

    // Cek apakah request <= available
    for (int j = 0; j < R; j++) {
        if (req[j] > available[j]) {
            printf("Resource tidak cukup. Proses harus menunggu.\n");
            return 0;
        }
    }

    // Coba alokasikan sementara
    for (int j = 0; j < R; j++) {
        available[j] -= req[j];
        alloc[p][j] += req[j];
        need[p][j] -= req[j];
    }
}

```

```

// Cek apakah aman
if (isSafe()) {
    printf("Request disetujui. Sistem dalam keadaan aman.\n");
    return 1;
} else {
    // Kembalikan
    for (int j = 0; j < R; j++) {
        available[j] += req[j];
        alloc[p][j] -= req[j];
        need[p][j] += req[j];
    }
    printf("Request ditolak! Sistem menjadi tidak aman.\n");
    return 0;
}
}

int main() {
    printf("=== Banker's Algorithm ===\n");

    // Input Max matrix
    printf("\nMasukkan matriks MAX (%d proses × %d resource):\n",
P, R);
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            scanf("%d", &max[i][j]);

```



```

// Input Allocation matrix

printf("\nMasukkan matriks ALLOCATION (%d proses × %d
resource):\n", P, R);

for (int i = 0; i < P; i++)
    for (int j = 0; j < R; j++)
        scanf("%d", &alloc[i][j]);

// Hitung Need = Max - Allocation
for (int i = 0; i < P; i++)
    for (int j = 0; j < R; j++)
        need[i][j] = max[i][j] - alloc[i][j];

int choice;
while (1) {
    printf("\nMenu:\n1. Request Resource\n2. Cek Safe State\n3.
Exit\nPilihan: ");
    scanf("%d", &choice);

    if (choice == 1) {
        int p, req[R];
        printf("Masukkan nomor proses (0-%d): ", P-1);
        scanf("%d", &p);

        printf("Masukkan request %d resource: ", R);

```

```
    for (int j = 0; j < R; j++)
        scanf("%d", &req[j]);

    requestResource(p, req);
}
else if (choice == 2) {
    if (isSafe())
        printf("Sistem dalam keadaan AMAN.\n");
    else
        printf("Sistem TIDAK AMAN.\n");
}
else if (choice == 3) {
    break;
}
else {
    printf("Pilihan tidak valid!\n");
}
}

return 0;
}
```

```
root@RIDHO:~# nano banker.c
root@RIDHO:~# gcc banker.c -o banker
root@RIDHO:~# ./banker
=== Banker's Algorithm ===
```

Masukkan matriks MAX (4 proses × 3 resource):

```
7 5 3
3 2 2
9 0 2
2 2 2
```

Masukkan matriks ALLOCATION (4 proses × 3 resource):

```
0 1 0
2 0 0
3 0 2
1 0 1
```

Menu:

1. Request Resource
2. Cek Safe State
3. Exit

Pilihan: 2

Sistem dalam keadaan AMAN.

Menu:

1. Request Resource
2. Cek Safe State
3. Exit

Pilihan: 1

Masukkan nomor proses (0-3): 1

Masukkan request 3 resource: 1 0 2

Request disetujui. Sistem dalam keadaan aman.

Menu:

1. Request Resource
2. Cek Safe State
3. Exit

Pilihan: 1

Masukkan nomor proses (0-3): 0

Masukkan request 3 resource: 5 2 3

Resource tidak cukup. Proses harus menunggu.

```
Menu:
1. Request Resource
2. Cek Safe State
3. Exit
Pilihan:
3
root@RIDHO:~# |
```

1. Available = barang yang masih tersedia

Contoh:

Available = [5, 4, 3]

Artinya:

- Ada 5 barang A yang masih bebas
- Ada 4 barang B
- Ada 3 barang C

2. Max = jumlah barang maksimal yang dibutuhkan setiap proses

Ini seperti pelanggan bilang:

“Saya maksimal butuh barang sebanyak ini.”

3. Allocation = barang yang sudah dipinjam proses saat ini

Ini seperti pelanggan sudah meminjam beberapa barang.

4. Need = barang yang masih dibutuhkan

Rumusnya simpel:

Need = Max – Allocation

5. Proses meminta barang (Request)

Jika proses minta barang:

Program harus cek dulu:

- Apakah yang diminta $\leq$ Need?

(Artinya: tidak lebih dari yang dia butuhkan)

- Apakah yang diminta $\leq$ Available?

(Artinya: barang cukup untuk dipinjamkan)

Kalau salah satu tidak cukup $\rightarrow$ langsung ditolak.

6. Coba berikan barang secara sementara

Kalau request valid, program coba memberikan barang sementara lalu mengecek:

Apakah sistem masih aman?

Atau malah bisa menyebabkan deadlock?

7. Safe State = tidak terjadi deadlock

Sistem aman jika:

- Semua proses masih punya kesempatan untuk selesai secara bergiliran.

Jika setelah request sistem tidak aman,
request ditolak, barang dikembalikan.

TUGAS 4

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* transferA(void* arg) {
```

```
    printf("A: Transfer dari Rekening 1 ke 2\n");
```

```
    // Lock selaluurut: 1 -> 2
```

```
    pthread_mutex_lock(&rekening1);
```

```
    printf("A: Mengunci Rekening 1\n");
```

```
    sleep(1);
```

```
    pthread_mutex_lock(&rekening2);
```

```
    printf("A: Mengunci Rekening 2\n");
```

```
    printf("A: Transfer berhasil!\n");
```

```
    pthread_mutex_unlock(&rekening2);
```

```
    pthread_mutex_unlock(&rekening1);
```

```
    return NULL;
}

void* transferB(void* arg) {
    printf("B: Transfer dari Rekening 2 ke 1\n");

    // Lock tetapurut: 1 -> 2 (meski sumbernya rekening 2)
    pthread_mutex_lock(&rekening1);
    printf("B: Mengunci Rekening 1\n");

    sleep(1);

    pthread_mutex_lock(&rekening2);
    printf("B: Mengunci Rekening 2\n");

    printf("B: Transfer berhasil!\n");

    pthread_mutex_unlock(&rekening2);
    pthread_mutex_unlock(&rekening1);

    return NULL;
}

int main() {
    pthread_t tA, tB;
```



```
printf("=====\n");
printf(" PREVENTION: TANPA DEADLOCK \n");
printf("=====\n");

pthread_create(&tA, NULL, transferA, NULL);
pthread_create(&tB, NULL, transferB, NULL);

pthread_join(tA, NULL);
pthread_join(tB, NULL);

printf("Selesai tanpa deadlock!\n");
return 0;
}
```

```
root@RIDHO:~# nano transfer_deadlock.c
root@RIDHO:~# gcc transfer_deadlock.c -o transfer -lpthread
root@RIDHO:~# ./transfer
=====
      PREVENTION: TANPA DEADLOCK
=====
A: Transfer dari Rekening 1 ke 2
A: Mengunci Rekening 1
B: Transfer dari Rekening 2 ke 1
A: Mengunci Rekening 2
A: Transfer berhasil!
B: Mengunci Rekening 1
B: Mengunci Rekening 2
B: Transfer berhasil!
Selesai tanpa deadlock!
root@RIDHO:~# |
```

- Tugas ini mengajarkan:

Mengapa deadlock terjadi

dan bagaimana mencegahnya dalam program nyata seperti transfer rekening.

KESIMPULAN

Selama praktikum ini, mahasiswa mempelajari bahwa deadlock merupakan masalah penting yang sering terjadi pada sistem operasi maupun aplikasi dunia nyata. Deadlock dapat membuat proses berhenti total karena ada dua atau lebih proses yang saling menunggu resource satu sama lain. Deadlock hanya terjadi apabila empat kondisi terpenuhi, yaitu mutual exclusion, hold and wait, no preemption, dan circular wait. Oleh karena itu, untuk membuat sistem berjalan dengan aman, keempat kondisi ini harus dicegah.

Melalui berbagai tugas, mahasiswa belajar beberapa teknik pencegahan deadlock. Pada tugas analisis, mahasiswa memahami bagaimana deadlock terbentuk dan memvisualisasikannya melalui resource allocation graph. Pada tugas pencegahan, mahasiswa memodifikasi program untuk menghindari deadlock menggunakan teknik lock ordering dan try-lock dengan retry. Mahasiswa juga belajar membandingkan efisiensi kedua metode tersebut.

Selain itu, pada tugas Banker's Algorithm, mahasiswa memahami bagaimana sistem dapat memutuskan apakah pemberian resource aman atau tidak. Algoritma ini memastikan sistem tetap berada dalam safe state sebelum memberikan resource tambahan sehingga deadlock dapat dihindari. Pada tugas kasus nyata, mahasiswa menerapkan konsep deadlock dan pencegahannya pada simulasi transfer rekening, menunjukkan bahwa masalah deadlock bukan hanya teori, tetapi juga terjadi pada sistem yang sering digunakan dalam kehidupan sehari-hari.

Secara keseluruhan, praktikum ini memberikan pemahaman yang lebih dalam tentang bagaimana deadlock terjadi, bagaimana menganalisisnya, dan bagaimana mencegahnya dengan teknik-teknik yang tepat agar sistem tetap berjalan stabil dan aman.

Saran

Untuk menghindari deadlock pada implementasi nyata, penggunaan lock harus dilakukan dengan teratur dan konsisten. Salah satu cara efektif adalah menerapkan aturan lock ordering, yaitu selalu mengunci resource dalam urutan tertentu agar tidak terjadi circular wait. Selain itu, disarankan untuk tidak memegang lock terlalu lama sehingga resource dapat segera digunakan proses lain. Pada sistem yang membutuhkan banyak proses berjalan secara bersamaan, penggunaan mekanisme try-lock juga dapat membantu karena proses tidak akan menunggu terlalu lama jika resource sedang digunakan.

Algoritma Banker juga dapat diterapkan pada sistem dengan banyak proses dan resource terbatas untuk memastikan alokasi resource selalu berada dalam kondisi aman. Selain itu, penggunaan diagram atau visualisasi seperti resource allocation graph sangat membantu untuk mendeteksi pola deadlock lebih cepat. Dengan menerapkan teknik-teknik ini secara konsisten, sistem dapat berjalan lebih stabil dan resiko terjadinya deadlock dapat diminimalkan.